

STAT3888 自行车盗窃

时空基函数回归

从时空可视化到可信预测的完整项目手册

数据	bicycle.csv · 31,833条事件 · 140个社区 · 2014–2023
主线	Task 1 时空可视化 + Task 4 基函数线性回归
基础	从高等代数、概率论进入设计矩阵、样条、验证和诊断
版本	2026-07-27 · 基于真实数据验证结果

这本手册强调理解、复现与诚实验证。所有模型指标均来自真实的2023时间外测试集。

目录

章节按“核心路线—进阶加分—未来连接”组织。先完成核心路线，再根据时间深入。

目录	2
如何使用这本手册	4
项目要求、交付物与评分策略	4
四个Task的递进关系	4
Task 1：时空可视化是什么	5
Task 2：EOF分析是什么	5
Task 3：IDW预测是什么	6
Task 4：基函数线性回归是什么	6
为什么选择 bicycle.csv	7
研究问题、分析单位与目标变量	7
从高等代数和概率论进入模型	7
线性回归与设计矩阵	8
基函数：把一个变量变成一组可学习特征	8
B-spline：学习平滑的长期趋势	9
Fourier基函数：表示年度季节性	9
径向基函数：表示连续空间热点	9
社区指示基函数：学习稳定地点差异	10
Ridge正则化与偏差—方差权衡	10
为什么不能随机拆分时空数据	10
第0步：连接真实分析管道并检查环境	11
第1步：解析数据路径并读取事件表	11
第2步：运行正式数据审计	12
第3步：调用正式函数构造社区 × 月份数据	13
第4步：检查零事件月份、坐标和时间索引	13
第5步：用正式函数生成 Task 1 四张图	14
第6步：按时间划分训练、验证和测试集	15
第7步：构造同一套时空基函数设计矩阵	16
第8步：一次拟合全部基线、OLS 和 Ridge	16
第9步：检查 OLS 基函数预测	17
第10步：检查 Ridge 的 λ 选择和最终预测	18
第11步：读取统一的 MAE、RMSE 和 R^2	19
第12步：从正式预测对象生成诊断图	19
第13步：把真实证据压缩为四页 PowerPoint	20
第14步：从干净会话一键复现并核对逐步结果	20

真实模型结果与解释	21
预测图、残差图与模型诊断	21
四页PPT逐页设计	23
答辩问题与推荐回答	23
局限性、伦理与不可推广范围	23
从本项目走向未来数据科学	24
从开始到提交的最终检查清单	24
资料与继续学习入口	24

核心路线

如何使用这本手册

这不是一份“复制代码就交作业”的答案，而是一条从已有数学基础进入时空数据科学的完整路线。第一次阅读先理解研究问题和四个Task；第二次按操作章节运行R；第三次根据真实输出制作四页PPT。

- 核心路线：项目必须理解和执行。
- 进阶加分：用于体现方法深度，但必须建立在核心路线正确之上。
- 未来连接：帮助你把本项目迁移到GAM、树模型和时空机器学习，现在不要求全部掌握。

五个检查问题

面对任何公式或代码，先问：数据是什么形状？输入是什么？输出是什么？模型相信什么结构？怎样用未见数据证明结果可信？

核心路线

项目要求、交付物与评分策略

原项目要求从四个数据集中选择一个，完成必做的Task 1时空可视化，再从Task 2 EOF、Task 3 IDW、Task 4基函数线性回归中选择一个。最终提交最多四页PowerPoint。

时空可视化	30%	图回答明确问题；时间、空间、颜色尺度和单位完整
选修方法	40%	原理正确；验证无泄漏；与基线比较；诚实解释局限
表达与逻辑	30%	研究问题贯穿数据、方法、结果和结论

最重要的策略

不要因为方法名字高级就堆模型。评分最高的路线是：研究问题清楚、方法和数据匹配、验证方式可信、四页之内讲得明白。

核心路线

四个Task的递进关系

Task 1是所有路线的共同起点：先看见规律和问题。三个选修Task分别把探索发展为模式分解、空间插值或解释与预测。它们没有绝对的新旧高低，关键是是否回答了正确的问题。

Task 1	数据在什么地方、什么时候发生什么变化？	地图、时间序列、热力图
Task 2	哪些共同空间模式解释了大部分变化？	EOF载荷、PC时间序列、方差比例
Task 3	没有观测的位置可能是多少？	IDW插值面、交叉验证误差
Task 4	时空结构能否解释并预测结果？	模型、预测、残差和测试指标

本项目的选择

选择Task 1 + Task 4，因为bicycle数据具有十年时间、固定社区位置和明确的未来预测问题。

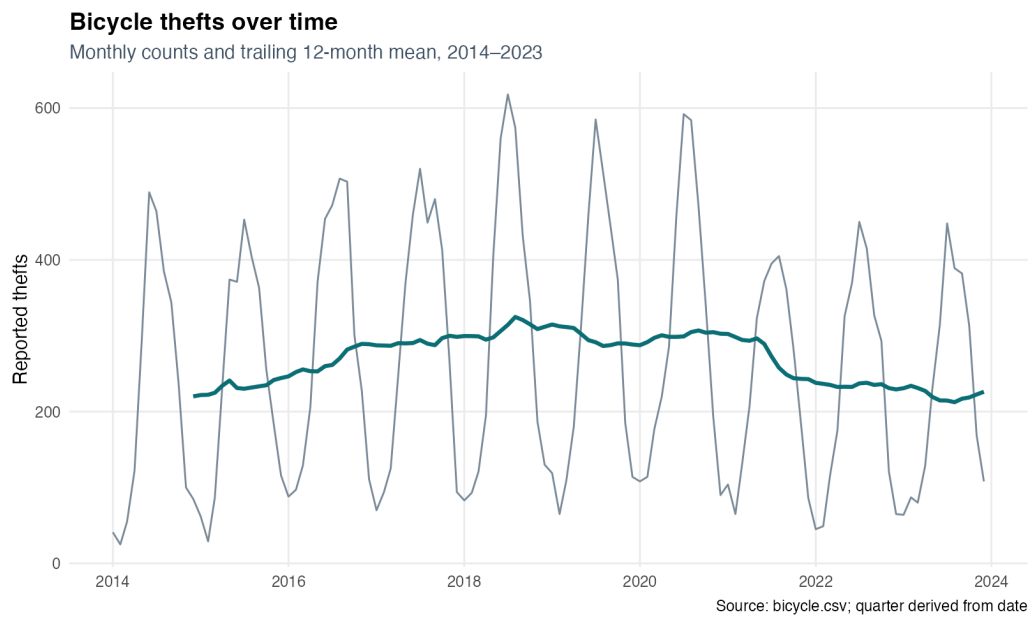
核心路线

Task 1：时空可视化是什么

时空可视化不是把经纬度画成点，而是同时说明“在哪里、什么时候、数值多大、怎样变化”。它用于发现热点、趋势、季节性、异常、缺失和尺度问题，也是后续模型设计的证据。

时间序列	总体数量怎样随时间变化？	每月盗窃数与12个月移动趋势
季节箱线图	同一月份跨年份是否稳定？	一月至十二月分布
空间热点图	长期高值集中在哪里？	社区中心点大小和颜色
时空热力图	地点模式怎样随时间同步变化？	前25社区 × 月份

- 比较多张地图时必须固定颜色范围。
- 0表示记录数为零；NA表示没有观测，两者不能混淆。
- 热点是描述性结果，不等于该社区本身导致盗窃。
- 经纬度是社区中心点，不是每起案件的准确位置。



Task 1 示例：总体月度趋势与12个月移动平均。

完整解释

Task 2：EOF分析是什么

EOF把“时间 × 地点”矩阵压缩为少数主要空间模式和对应的的时间变化。它等价于对中心化的时空矩阵做PCA/SVD，最适合规则、较完整的空间面板。

X的行是时间、列是地点；V的每一列给出一个EOF空间模式；U Σ 给出对应PC时间序列； Σ 中的奇异值决定解释方差比例。

- 中心化：减去每个地点的时间均值，关注偏离正常水平的共同变化。
- 标准化：只有在希望每个地点拥有相同权重时使用；否则会放大低方差地点。

- EOF正负号可以整体翻转，空间图和PC同时翻转后含义不变。
- EOF1解释方差最大，不等于因果最重要，也不自动具备预测能力。

适合的数据

COVID地点×日期或汇总后的bicycle社区×月份都可构造矩阵；缺失严重的Big Mac面板需要先处理缺期。

完整解释

Task 3 : IDW预测是什么

IDW (Inverse Distance Weighting) 根据附近已观测地点预测未观测地点。核心假设是空间上越近的地点越相似，因此距离较近的观测获得更大权重。

s 是目标地点， s_i 是已观测地点， d_i 是距离， p 控制距离衰减速度。 p 越大，预测越依赖最近地点。

- 必须留出部分已知地点，假装它们未知，再用RMSE/MAE评价。
- p 和邻居数量应通过交叉验证选择，而不是凭感觉。
- 大范围经纬度不能简单当平面坐标距离；需要投影或球面距离。
- IDW通常不给可靠不确定性，也不使用协方差结构。
- 自行车盗窃可能受商业区、交通和暴露量影响，不一定只由距离决定，因此不作为本项目主方法。

核心路线

Task 4 : 基函数线性回归是什么

普通直线只能表示固定斜率。基函数先把时间和空间坐标转换成一组更丰富的特征，再对这些特征做线性回归。模型对输入可以是弯曲、周期和局部的，但对待估系数仍然是线性的。

y 是社区 s 在月份 t 的盗窃数。 B_{trend} 表示长期趋势， B_{season} 表示季节周期， B_{space} 表示空间和社区差异。

B-spline	平滑且可局部弯曲	十年长期趋势
Fourier	周期性的正弦和余弦	每12个月重复的季节性
径向基函数	围绕空间中心逐渐衰减	连续空间热点
社区指示基函数	每个社区独立基线	稳定的社区差异

为什么选它

它严格符合Task 4，同时训练特征工程、设计矩阵、验证、正则化、诊断和解释，能够连接未来的数据科学学习。

核心路线

为什么选择 bicycle.csv

bicycle	31,833事件；140社区；10年；无缺失	重复行无ID；坐标为社区中心	最适合
covid	规则日期；可直接形成面板	仅51天；累计值修订；地理层级混杂	次选
bigmac	全球题材；连续价格	面板完整率约70.7%；日期歧义	清洗成本高
ufo	题材醒目；有城市坐标	日期歧义；报告偏差；事件稀疏	不推荐

bicycle的固定社区位置允许学习空间结构，120个月允许学习季节和长期趋势。汇总为社区×月份后，共有16,800个分析单元，足以比较简单基线与基函数模型。

不分析bike_cost的原因

核心问题是盗窃数量的时空变化。bike_cost包含1,623个非正值和少量极端值，加入会改变研究问题并增加不必要清洗。

核心路线

研究问题、分析单位与目标变量

研究问题：时空基函数能否预测不同社区的自行车盗窃数量，并揭示盗窃活动随空间、季节和长期时间的变化？

- 原始粒度：一行是一条被报告的盗窃事件。
- 建模粒度：一行是一个社区在一个月份的盗窃数量。
- 空间单位：140个社区及其中心经纬度。
- 时间单位：2014年1月至2023年12月，共120个月。
- 响应变量：theft_count；为减弱右偏，模型使用 $\log(1+\text{count})$ 。

解释边界

没有人口、自行车拥有量、通勤量或报案概率，因此结果描述的是“报告数量”，不能称为个人风险或因果效应。

核心路线

从高等代数和概率论进入模型

你已经学过的向量、矩阵、线性变换、期望、方差和协方差不是旁支，而是这个项目的语言。每个社区月份形成一行特征向量，所有行叠成设计矩阵 X ；系数向量 β 表示每个特征的权重。

X 是 $n \times p$ 设计矩阵， $n=16,800$ 个社区月份， p 是基函数个数； β 是 $p \times 1$ 系数； ϵ 是模型未解释的随机误差。

- 矩阵乘法：把多个基函数特征按系数组合成预测。
- 秩与共线性：列过于相似时，系数不稳定。
- 期望：模型预测可理解为给定特征后的平均水平。
- 方差：衡量结果波动，也连接到RMSE和偏差-一方差权衡。
- 协方差：解释相邻月份或相邻地点为什么可能具有相关误差。

核心路线

线性回归与设计矩阵

“线性”指模型对系数 β 是线性的，而不要求输入变量只能原样进入。只要先计算好特征列，平方、正弦、样条或空间核都能成为 X 的一列。

最小二乘选择使残差平方和最小的系数。残差是实际值减预测值，不是原数据的误差标签。

y	16.800×1	每个社区月份的 $\log(1+\text{盗窃数})$
X	$16.800 \times p$	时间、季节、空间和社区基函数
β	$p \times 1$	每一列基函数的待估权重
$\hat{y}$	16.800×1	模型预测，再用 $\exp(\hat{y})-1$ 还原

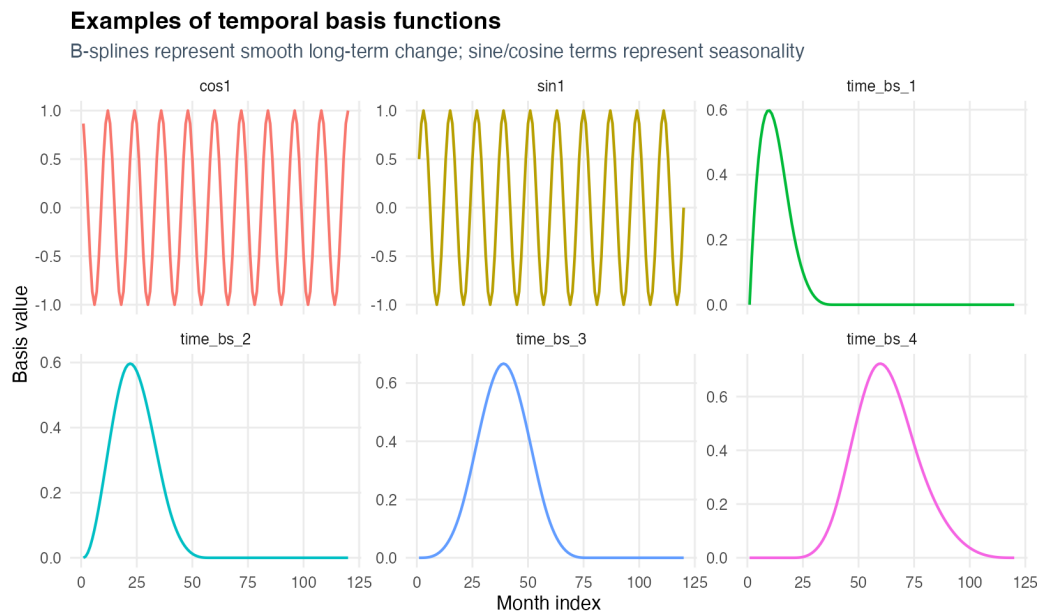
核心路线

基函数：把一个变量变成一组可学习特征

基函数 $\phi(x), \dots, \phi_K(x)$ 像一组积木。每一块有固定形状，模型学习怎样加权组合。这把“拟合一条复杂曲线”转化成“估计一组线性系数”。

ϕ_k 由我们选择， β_k 由数据估计。K太少会欠拟合，K太多可能过拟合，因此需要验证或正则化。

- 好处：非线性仍能使用成熟的线性代数和回归工具。
- 代价：基函数类型、数量和节点位置成为新的建模选择。
- 原则：每类基函数对应明确结构，而不是无限添加列。



项目中的B-spline与Fourier基函数示例。

核心路线

B-spline：学习平滑的长期趋势

高次多项式在整个区间相互牵动，边界容易剧烈震荡。B-spline把时间区间分成若干段，每个基函数只在局部区域显著，因此既能弯曲又相对稳定。节点knots决定不同多项式片段连接的位置。

R代码

```
time_basis <- splines::bs(
  panel$time_index,
  knots = c(20, 39, 58, 77),
  Boundary.knots = c(1, 120),
  degree = 3
)
```

- degree=3表示三次分段多项式。
- 节点从训练期确定；边界使用已知的项目月份范围，不使用未来盗窃值。
- 模型学习基函数权重，不直接把单个系数解释为某月增长率。
- 长期趋势图应展示组合后的效应，而不是逐个系数。

核心路线

Fourier基函数：表示年度季节性

自行车盗窃具有明显年度周期。月份类别需要11个参数且在十二月与一月之间不连续；正弦和余弦天然首尾相接，用少量参数表示平滑周期。

m是一月至十二月。第一对表示一年一次的波动，第二对允许一年内出现更复杂的双峰或不对称形状。

R代码

```
sin1 <- sin(2*pi*month_of_year/12)
cos1 <- cos(2*pi*month_of_year/12)
sin2 <- sin(4*pi*month_of_year/12)
cos2 <- cos(4*pi*month_of_year/12)
```

与概率论的连接

这里不是假设盗窃服从正弦分布，而是把周期函数当作确定的特征列；随机性仍留在误差 ϵ 中。

核心路线

径向基函数：表示连续空间热点

径向基函数以某个空间中心为峰值，距离越远取值越小。多个中心的函数叠加后，可以形成平滑的空间表面。它比直接加入经纬度直线项更能表达局部热点。

s是社区坐标， ck 是第 k 个空间中心， σ 控制影响范围。 σ 太小会形成尖峰，太大会把城市抹平。

- 本项目从社区中心中均匀选取16个RBF中心。
- 经纬度先中心化并按共同尺度缩放，避免数值单位影响。
- RBF表达连续位置，但坐标仍只是社区中心。
- 如果目标是新城市或城市外推，当前基函数不保证可靠。

进阶加分

社区指示基函数：学习稳定地点差异

第一版只使用连续空间RBF时，模型没有超过社区历史均值。原因是两个经纬度接近的社区仍可能因土地用途、商业活动或报案习惯而长期不同。于是加入每个社区的一系列0/1指示基函数。

如果观测属于社区 j ，该列为1，否则为0。这允许模型学习每个社区的稳定基线，同时保留季节和长期趋势。

重要取舍

社区指示基函数提升已知社区的预测，但无法直接预测训练中从未出现的新社区。因此它服务于“预测2023年现有140社区”，空间留出时应另用RBF-only模型做敏感性检查。

进阶加分

Ridge正则化与偏差一方差权衡

基函数增多后，部分列可能高度相关。Ridge在残差平方和之外惩罚大系数，使解更稳定。惩罚强度 λ 必须用验证集选择，不能看测试集后再调整。

$\lambda=0$ 退化为普通最小二乘； λ 越大，系数越接近0，模型越平滑但偏差可能增加。

真实结果中，2022验证选择的 λ 约为0.0001，接近无惩罚；2023测试RMSE中Basis OLS为2.05、Basis Ridge为2.06。这说明正则化作为稳健性工具有价值，但不能宣称它在本数据上优于OLS。

现代数据科学不是强迫复杂模型胜出

正确结论是：验证决定复杂度，测试集只做一次最终评价；如果简单模型更好，就选择或报告简单模型。

核心路线

为什么不能随机拆分时空数据

随机拆行会让同一社区的2023年1月进入训练、2月进入测试，模型等于提前看到了未来附近的信息。这会低估真实部署误差，属于数据泄漏。

训练集	2014–2021	估计模型与基础结构
验证集	2022	选择 λ 等超参数
测试集	2023	一次性报告最终泛化性能

- 时间验证始终用过去预测未来。
- 所有均值、节点、缩放参数和系数只能由训练数据得到。
- 测试集结果不好时不能返回修改模型再重测，否则测试集变成了验证集。
- 未来可使用rolling/sliding validation观察不同时段稳定性。

核心路径· 照做与理解

第0步：连接真实分析管道并检查环境

这一步解决什么问题	让你当前的 RStudio Project 调用已经验证的正式函数，同时仍把输出保存在当前 Project。
为什么现在运行	以前 PDF 重新写了一套简化代码，容易与正式结果分叉。现在先加载唯一真实管道，后面每一步只调用这些函数。
输入对象	当前工作目录应为 `/Users/liumingyuan/stat3888`；正式代码位于 `/Users/liumingyuan/Documents/project of stat3888`。
可直接运行	<pre>getwd() project_code_root <- "/Users/liumingyuan/Documents/project of stat3888" source(file.path(project_code_root, "R", "handbook_step_by_step.R")) workspace_root <- getwd() paths <- project_paths(workspace_root) ensure_packages() paths\$source_csv</pre>
逐行解释	<code>`getwd()`</code> 返回当前 RStudio Project。<code>`project_code_root`</code> 只告诉 R 正式函数放在哪里；<code>`source()`</code> 把这些函数加载进当前会话。<code>`workspace_root <- getwd()`</code> 决定分析 CSV、图片和结果保存在哪里。<code>`project_paths(workspace_root)`</code> 调用 <code>`resolve_bicycle_csv()`</code> 寻找真实数据，而不再依赖一个失效的绝对文件名。<code>`ensure_packages()`</code> 只检查包，不改变数据。
运行后应该看到	控制台显示你当前 Project 的路径和一个确实存在的 bicycle CSV。Environment 中出现 <code>`run_handbook_steps`</code> 等函数。

检查命令

```
stopifnot(dir.exists(workspace_root))
stopifnot(file.exists(paths$source_csv))
all(c("tidyverse", "lubridate", "glmnet") %in%
  rownames(installed.packages()))
```

常见错误	若 <code>`source()`</code> 找不到文件，检查 <code>`project_code_root`</code> 是否原样复制。若提示缺少软件包，只对缺失包运行 <code>`install.packages(c("包名"))`</code> ，安装后重新执行 <code>`ensure_packages()`</code> 。
------	--

评分连接	清楚区分“代码仓库”和“分析工作目录”，能够避免路径错误并证明项目可复现。
------	---------------------------------------

核心路径· 照做与理解

第1步：解析数据路径并读取事件表

这一步解决什么问题	读取与最终报告完全相同的 bicycle CSV，并用正式读取函数创建 <code>`raw`</code> 。
为什么现在运行	数据副本或路径不同会让后续所有数字不一致，所以第一步必须显示实际读取的是哪个文件。
输入对象	<code>`paths\$source_csv`</code> ：路径解析器找到的现存文件；每行是一条被报告的盗窃事件。

可直接运行

```
data_path <- resolve_bicycle_csv(workspace_root)
raw <- read_bicycle(data_path)
original_columns <- c(
  "date", "quarter", "day_of_week", "neighborhood",
  "bike_cost", "location", "long", "lat"
)
c(rows = nrow(raw), original_columns = length(original_columns))
dplyr::glimpse(raw[, original_columns])
```

逐行解释

`resolve\_bicycle\_csv()` 的优先级是环境变量、当前 Project 的 `data/bicycle.csv`、你现有的 `studio/data` 和下载目录副本。`raw <-` 把 `read\_bicycle()` 的返回值保存为对象；`<-` 表示赋值。该函数内部使用 `readr::read\_csv(..., show\_col\_types = FALSE)`；`readr::` 指明函数来源，`show\_col\_types` 只关闭提示。正式 `raw` 还增加四个季度审计辅助列，因此检查原始字段时显式选择 `original\_columns`。

运行后应该看到

显示 31,833 行、8 个原始字段；日期范围为 2014-01-01 至 2023-12-31。

检查命令

```
stopifnot(nrow(raw) == 31833L)
stopifnot(all(original_columns %in% names(raw)))
range(raw$date)
data_path
```

常见错误

若行数不是 31,833，立即停止并查看 `data\_path`。不要通过改期望数字让检查通过；先确认没有读取其他城市或修改后的 CSV。

评分连接

明确数据版本、观测单位和样本量，是数据介绍与全部结果可信性的起点。

核心路径·照做与理解

第2步：运行正式数据审计

这一步解决什么问题

用与一键管道相同的 `audit\_bicycle()` 统计缺失、重复、社区数和季度不一致。

为什么现在运行

审计必须先于删除。没有事件 ID 时，完全相同行不一定是重复录入。

输入对象

`raw`：包含八个原始字段和正式读取函数生成的季度辅助字段。

可直接运行

```
audit <- audit_bicycle(raw)
audit
```

逐行解释

`audit\_bicycle()` 内部使用 `is.na()` 统计缺失、`duplicated()` 标记与前行相同的记录、`n\_distinct()` 统计社区。它比较从 `date` 推导的季度与原 `quarter`，但不修改原始文件，也不执行 `distinct()`。

运行后应该看到

缺失单元格 0；完全重复行 1,475；社区 140；quarter 不一致 15,751。

检查命令

```
stopifnot(audit$value[audit$metric == "rows"] == 31833)
stopifnot(audit$value[audit$metric == "exact_duplicate_rows"] == 1475)
stopifnot(audit$value[audit$metric == "neighborhoods"] == 140)
```

常见错误

不要对 `raw` 运行 `distinct()` 后再继续，否则盗窃计数、所有图和模型指标都会改变。季度分析使用 `date` 推导值。

评分连接

高分数据清洗强调有证据的决定：保留不确定重复记录，并明确说明理由。

函数内部原理展开（不单独运行）

```
raw |>
  dplyr::mutate(
    date = as.Date(date),
    derived_quarter = paste0(
      lubridate::year(date), "Q",
      lubridate::quarter(date)
    )
  )
```

核心路径·照做与理解

第3步：调用正式函数构造社区 × 月份数据

这一步解决什么问题 把事件表转换成统一的社区月份分析单位，但不另写第二套 `count()` 逻辑。

为什么现在运行 Task 1 和 Task 4
必须共享同一个面板。调用正式函数可以保证逐步路线与一键路线不会再次漂移。

输入对象 `raw`；输出 `prepared`，其中同时包含完整面板和社区中心坐标表。

可直接运行

```
prepared <- make_monthly_panel(raw)
panel <- prepared$panel
coordinates <- prepared$coordinates
dim(panel)
names(panel)
```

逐行解释 `make\_monthly\_panel(raw)` 是唯一面板构造入口。`prepared\$panel` 用 `\$` 从列表取出面板；`prepared\$coordinates` 保存每个社区的经纬度。`dim()` 返回行列数，`names()` 检查列是否完整。

运行后应该看到 `panel` 为 16,800 × 8，包含 neighborhood、month、theft_count、lon、lat、time_index、month_of_year、year。

检查命令

```
stopifnot(nrow(panel) == 16800L)
stopifnot(all(c("lon", "lat") %in% names(panel)))
stopifnot(nrow(coordinates) == 140L)
```

常见错误 若只有 6 列，说明你运行了旧 PDF 的手工 `complete()` 代码，而没有调用 `make\_monthly\_panel(raw)`。请删除旧 panel 并重新执行本步。

评分连接 统一面板同时支撑时间、空间可视化和模型，是方法一致性的核心证据。

函数内部原理展开（不单独运行）

```
monthly_counts <- raw |>
  dplyr::mutate(
    month = lubridate::floor_date(date, "month")
  ) |>
  dplyr::count(
    neighborhood, month, name = "theft_count"
  )
```

为什么使用管道 `|>` 把左侧对象传给右侧函数；`mutate()` 新增月份；`count()` 按社区和月份计数。这段展示正式函数内部原理，不应另行生成替代 panel。

核心路径·照做与理解

第4步：检查零事件月份、坐标和时间索引

这一步解决什么问题 证明正式面板既补齐了 0，又保留了空间坐标和建模所需时间字段。

为什么现在运行	旧 PDF 只补齐计数却漏掉 lon/lat，导致 RBF 和地图无法运行。本步专门阻止该错误再次出现。
输入对象	`panel`：第 3 步由正式函数生成的 16.800 行面板。
可直接运行	<pre>panel > dplyr::summarise(rows = dplyr::n(), zero_months = sum(theft_count == 0), missing_counts = sum(is.na(theft_count)), missing_coordinates = sum(is.na(lon) is.na(lat)), first_index = min(time_index), last_index = max(time_index))</pre>
逐行解释	`summarise()` 把多行压缩为检查摘要；`n()` 统计行数。0 表示该社区月份没有记录，NA 表示未知，两者不能混淆。`time_index` 把 2014-01 至 2023-12 编为 1-120。
运行后应该看到	16.800 行；计数和坐标均无缺失；time_index 范围为 1-120；每个社区 120 个月。
检查命令	<pre>stopifnot(!anyNA(panel\$theft_count)) stopifnot(!anyNA(panel\$lon), !anyNA(panel\$lat)) stopifnot(all(table(panel\$neighborhood) == 120L)) stopifnot(identical(range(panel\$time_index), c(1L, 120L)))</pre>
常见错误	若坐标缺失，不要手工用 0 填充；回到第 3 步确认使用正式函数。经纬度为社区中心，不是案件精确地点。
评分连接	检查空间字段与零月份可避免错误图和错误设计矩阵进入展示。

函数内部原理展开（不单独运行）

```
tidyr::complete(
  neighborhood = unique(raw$neighborhood),
  month = months,
  fill = list(theft_count = 0L)
) |>
dplyr::left_join(coordinates, by = "neighborhood")
```

核心路径·照做与理解

第5步：用正式函数生成 Task 1 四张图

这一步解决什么问题	生成与最终 PDF 和一键管道完全相同的时间、季节、空间和时空图。
为什么现在运行	Task 1 占 30%。每张图必须使用正确聚合口径，特别是季节图不能把 16.800 个社区月份误当成 120 个城市月份。
输入对象	`panel` 与 `paths\$figure_dir`；输出四个真实 PNG 路径。
可直接运行	<pre>task1_paths <- make_task1_plots(panel, paths\$figure_dir) task1_paths</pre>
逐行解释	`make_task1_plots()` 内部建立四个对象并保存图。时间图使用城市月总数；季节图先按 year 和 month_of_year 汇总，因此一月至十二月每组各有十个年份观测；空间图使用社区长期月均；热力图使用前 25 个高值社区。
运行后应该看到	trend、seasonality、hotspots、heatmap 四个路径均存在，输出目录出现四张 PNG。

检查命令

```
stopifnot(length(task1_paths) == 4L)
stopifnot(all(file.exists(task1_paths)))
names(task1_paths)
```

常见错误

若季节箱线图数值集中在 0-几起，说明仍在用旧代码直接画社区月份。正式季节图是全市月总数，应为数百量级。

评分连接

正确聚合让图真正回答趋势、季节、空间热点和时空持续性四个问题。

函数内部原理展开（不单独运行）

```
monthly <- panel |>
  dplyr::group_by(month) |>
  dplyr::summarise(
    thefts = sum(theft_count), .groups = "drop"
  )
seasonal <- panel |>
  dplyr::group_by(year, month_of_year) |>
  dplyr::summarise(
    thefts = sum(theft_count), .groups = "drop"
  )
hotspots <- panel |>
  dplyr::group_by(neighborhood, lon, lat) |>
  dplyr::summarise(
    mean_monthly = mean(theft_count), .groups = "drop"
  )
top_names <- hotspots |>
  dplyr::slice_max(mean_monthly, n = 25) |>
  dplyr::pull(neighborhood)
heat <- panel |>
  dplyr::filter(neighborhood %in% top_names)
```

ggplot

如何使用这些对象

`ggplot(data, aes(...))` 负责数据与坐标、颜色或大小的映射；`geom\_line()` 画时间线，`geom\_boxplot()` 画季节分布，`geom\_point()` 画空间热点，`geom\_tile()` 画时空热力图。

核心路径·照做与理解

第6步：按时间划分训练、验证和测试集

这一步解决什么问题

用过去估计、用 2022 选择设置、用完全未见的 2023 报告泛化结果。

为什么现在运行

随机拆行会让同一社区未来月份泄漏进训练，使结果虚高。

输入对象

`panel`，其中 year 已由 month 推导。

可直接运行

```
splits <- split_panel(panel)
train <- splits$train
validation <- splits$validation
test <- splits$test
vapply(splits, nrow, integer(1))
```

逐行解释

`split\_panel()` 内部使用 `filter()`：`year <= 2021`、`year == 2022`、`year == 2023`。`==` 是相等比较，单个 `=` 不能在这里替代它。列表中的三个对象互不重叠。

运行后应该看到

train 13,440 行；validation 1,680 行；test 1,680 行。

检查命令

```
stopifnot(max(train$year) < min(validation$year))
stopifnot(max(validation$year) < min(test$year))
stopifnot(sum(vapply(splits, nrow, integer(1))) == 16800L)
```

常见错误

不要使用 `sample()`；不要在看到 test 指标后修改模型并反复重测。

评分连接

无泄漏时间验证是 Task 4 可信度最重要的证据之一。

核心路径·照做与理解

第7步：构造同一套时空基函数设计矩阵

这一步解决什么问题

把时间、季节、位置和社区转换为模型可读取的数值列，并保证三个数据段列结构完全一致。

为什么现在运行

设计矩阵 X 是高等代数与回归的连接。节点、中心和因子水平必须只由训练集确定。

输入对象

`splits`；输出 `basis\_recipe` 和三个数值矩阵。

可直接运行

```
basis_recipe <- make_basis_recipe(splits$train)
x_train <- make_design_matrix(
  splits$train, basis_recipe
)
x_validation <- make_design_matrix(
  splits$validation, basis_recipe
)
x_test <- make_design_matrix(
  splits$test, basis_recipe
)
c(dim(x_train), dim(x_validation), dim(x_test))
```

逐行解释

`make\_basis\_recipe()` 保存 B-spline 节点、经纬度中心与尺度、16 个 RBF 中心和社区 levels。`make\_design\_matrix()` 对三个数据段复用同一 recipe；内部用 `model.matrix(~ neighborhood - 1)` 创建社区 0/1 列。

运行后应该看到

时间节点为 20、39、58、77；RBF 中心 16；三个矩阵列数和列名完全相同。

检查命令

```
stopifnot(identical(
  as.numeric(basis_recipe$time_knots), c(20, 39, 58, 77)
))
stopifnot(nrow(basis_recipe$rbf_centers) == 16L)
stopifnot(identical(colnames(x_train), colnames(x_test)))
```

常见错误

若矩阵列数不同，通常是分别重新计算社区 levels 或 recipe。不要从 validation/test 重新选择节点或 RBF 中心。

评分连接

公式、设计矩阵和代码一一对应，证明模型不是黑箱。

函数内部原理展开（不单独运行）

```
time_basis <- splines::bs(
  data$time_index,
  knots = recipe$time_knots,
  Boundary.knots = recipe$time_boundary,
  degree = 3
)
seasonal <- cbind(
  sin1 = sin(2*pi*data$month_of_year/12),
  cos1 = cos(2*pi*data$month_of_year/12)
)
neighborhood_basis <- stats::model.matrix(
  ~ neighborhood - 1, data = data
)
```

核心路径·照做与理解

第8步：一次拟合全部基线、OLS 和 Ridge

这一步解决什么问题

调用唯一正式建模函数，避免分别复制四套预测代码造成结果漂移。

为什么现在运行

所有比较必须使用同一测试行、同一反变换和同一评价指标。

输入对象 `splits` 和训练期 `basis\_recipe`。

可直接运行

```
model_fit <- fit_models(splits, basis_recipe)
baseline_predictions <- model_fit$test_predictions |>
  dplyr::select(
    month, neighborhood, actual,
    global_mean, neighborhood_mean
  )
dplyr::slice_head(baseline_predictions, n = 3)
```

逐行解释 `fit\_models()` 同时生成全局均值、社区历史均值、Basis OLS 和 Basis Ridge。这里仅从 `model\_fit\$test\_predictions` 读取已经计算好的基线，而不是重新实现均值连接逻辑。

运行后应该看到 `baseline\_predictions` 有 1,680 行；全局均值每行相同，社区均值随 neighborhood 改变。

检查命令

```
stopifnot(nrow(baseline_predictions) == nrow(test))
stopifnot(!anyNA(baseline_predictions))
```

常见错误 若重新手写基线时使用 2022 或 2023 均值，会产生泄漏。使用 `model\_fit` 可保证与一键管道一致。

评分连接 透明基线证明复杂模型是否真正带来额外价值。

函数内部原理展开（不单独运行）

```
global_prediction <- rep(
  mean(train$theft_count), nrow(test)
)
neighborhood_means <- train |>
  dplyr::group_by(neighborhood) |>
  dplyr::summarise(
    value = mean(theft_count), .groups = "drop"
  )
```

核心路径·照做与理解

第9步：检查 OLS 基函数预测

这一步解决什么问题 理解最小二乘如何把设计矩阵变成 2023 预测，同时读取正式模型已经生成的 OLS 结果。

为什么现在运行 OLS 是 Task 4 的直接实现，也是本数据上测试 RMSE 最低的模型。

输入对象 `model\_fit\$test\_predictions` 中的 `basis\_ols` 列。

可直接运行

```
ols_predictions <- model_fit$test_predictions |>
  dplyr::select(
    month, neighborhood, actual, basis_ols
  )
summary(ols_predictions$basis_ols)
```

逐行解释 `basis\_ols` 已由正式 `fit\_models()` 生成。模型先对 `log1p(theft\_count)` 拟合，再用 `expm1()` 返回计数尺度并截断负值。矩阵乘法 `%\*%` 对应高等代数中的 XB 。

运行后应该看到 1,680 个非负 OLS 预测；测试 MAE 约 1.008、RMSE 约 2.048、 R^2 约 0.757。

检查命令

```
stopifnot(nrow(ols_predictions) == 1680L)
stopifnot(all(is.finite(ols_predictions$basis_ols)))
stopifnot(all(ols_predictions$basis_ols >= 0))
```

常见错误 不要拿 log 尺度预测直接与原始计数计算误差；不要逐个解释大量基函数系数为因果效应。

评分连接

OLS 将 $y=XB+\epsilon$ 、R 代码和测试集预测连成可解释证据链。

函数内部原理展开（不单独运行）

```
fit_ols <- function(x, y) {
  fit <- stats::lm.fit(
    cbind(Intercept = 1, x), y
  )
  fit$coefficients
}
prediction <- pmax(
  0, expm1(cbind(Intercept = 1, x_test) %*% beta)
)
```

核心路径·照做与理解

第10步：检查 Ridge 的 λ 选择和最终预测

这一步解决什么问题

确认 Ridge 只用 2022 选择 λ ，随后用 2014–2022 重拟合并预测 2023。

为什么现在运行

旧 PDF 只算到 best_lambda，没有完成最终 Ridge 拟合与预测；现在所有结果都直接来自正式函数。

输入对象

`model\_fit`，包含 selected_lambda、validation_rmse、ridge_model 和最终 predicted 列。

可直接运行

```
model_fit$selected_lambda
model_fit$validation_rmse
ridge_predictions <- model_fit$test_predictions |>
  dplyr::select(
    month, neighborhood, actual, predicted
  )
summary(ridge_predictions$predicted)
```

逐行解释

`selected\_lambda` 是验证 RMSE 最小的候选值。`predicted` 是 Ridge 在 2023 的最终计数尺度预测。`glmnet(..., alpha=0)` 表示 Ridge；`standardize=TRUE` 统一特征尺度。

运行后应该看到

 λ 约 $1e-4$ ；Ridge 测试 MAE 约 1.006、RMSE 约 2.060、 R^2 约 0.754，略逊于 OLS RMSE。

检查命令

```
stopifnot(model_fit$selected_lambda >= 1e-4)
stopifnot(nrow(ridge_predictions) == 1680L)
stopifnot(all(ridge_predictions$predicted >= 0))
```

常见错误

不要用 test 选择 λ ；不要因为 Ridge 更新就宣布它获胜。本项目应如实报告 OLS 的 RMSE 略低。

评分连接

完整超参数选择、重拟合和时间外预测体现现代建模规范。

函数内部原理展开（不单独运行）

```
ridge_path <- glmnet::glmnet(
  x_train, log1p(train$theft_count),
  alpha = 0, lambda = lambda_grid,
  standardize = TRUE
)
selected_lambda <- lambda_grid[
  which.min(validation_rmse)
]
ridge_final <- glmnet::glmnet(
  x_train_validation, y_train_validation,
  alpha = 0, lambda = selected_lambda
)
```

核心路径· 照做与理解

第11步：读取统一的 MAE、RMSE 和 R^2

这一步解决什么问题 从正式模型对象读取同一 2023 测试集上的四模型指标。

为什么现在运行 不再手工重算部分模型，避免预测向量、尺度或测试行不一致。

输入对象 ``model_fit$metrics``。

可直接运行

```
metrics <- model_fit$metrics
metrics
```

逐行解释 ``MAE`` 是平均绝对误差；``RMSE`` 对大错惩罚更强；``R^2=1-SSE/SST`` 比较模型与测试均值基准。 R^2 不是预测正确百分比，负值表示比均值基准更差。

运行后应该看到 Global mean : 2.187/4.167/-0.00696 ; Neighborhood mean : 1.411/2.726/0.569 ; Basis OLS : 1.008/2.048/0.757 ; Basis Ridge : 1.006/2.060/0.754 。

检查命令

```
stopifnot(nrow(metrics) == 4L)
stopifnot(all(c("MAE", "RMSE", "R2") %in% names(metrics)))
stopifnot(metrics$RMSE[metrics$model == "Basis OLS"] <
  metrics$RMSE[metrics$model == "Neighborhood mean"])
```

常见错误 若数值不同，先检查是否删除过重复行、是否漏补 0、是否随机拆分、是否忘记反变换；不要先修改期望结果。

评分连接 统一指标表是 Task 4 结论页最重要的量化证据。

核心路径· 照做与理解

第12步：从正式预测对象生成诊断图

这一步解决什么问题 检查模型在哪些月份和社区系统性低估或高估。

为什么现在运行 旧 PDF 直接引用未创建的 ``test_predictions``；现在明确从 ``model_fit$test_predictions`` 取得。

输入对象 ``model_fit$test_predictions``、``panel``、``basis_recipe`` 和输出目录。

可直接运行

```
test_predictions <- model_fit$test_predictions
model_paths <- make_model_plots(
  model_fit, panel, basis_recipe, paths$figure_dir
)
diagnostic <- test_predictions |>
  dplyr::mutate(residual_check = actual - predicted)
model_paths
```

逐行解释 ``test_predictions`` 已包含 actual、predicted 和 residual。``make_model_plots()`` 生成基函数、模型比较、预测折线和残差地图。``mutate()`` 再算 residual_check 只用于验证正式 residual 的定义。

运行后应该看到 四张模型图存在；residual 与 actual-predicted 在数值容差内完全一致。

检查命令

```
stopifnot(all(file.exists(model_paths)))
stopifnot(max(abs(
  diagnostic$residual - diagnostic$residual_check
)) < 1e-10)
```

常见错误

正残差表示实际高于预测，即模型低估。不要把残差热点写成社区导致盗窃。

评分连接

诊断失败模式比只报告最好分数更能体现分析深度。

核心路径·照做与理解

第13步：把真实证据压缩为四页 PowerPoint

这一步解决什么问题

用已验证图和指标建立“问题→发现→方法→结论”的四页叙事。

为什么现在运行

项目最多四页；选择证据比堆叠全部代码更重要。

输入对象

`task1\_paths`、`metrics`、`model\_paths` 和局限性结论。

可直接运行

```
slide_plan <- tibble::tribble(
  ~page, ~question, ~evidence,
  1, "研究什么？", "数据、问题、流程",
  2, "发现什么？", "趋势、季节、热点",
  3, "怎样建模？", "基函数、时间切分、指标",
  4, "结论可信到哪里？", "预测、残差、局限"
)
slide_plan
```

逐行解释

`tribble()` 建立展示检查表；`~page` 等带波浪号的项定义列名。它不是生成 PPT，而是确保每页只回答一个问题。

运行后应该看到

得到 4×3 的页面计划。

检查命令

```
stopifnot(nrow(slide_plan) == 4L)
stopifnot(length(unique(slide_plan$page)) == 4L)
```

常见错误

不要把代码截图塞入 PPT；不要把 $R^2 0.757$ 写成 75.7% 预测正确。

评分连接

表达与逻辑占 30%，四页必须形成连续证据链。

核心路径·照做与理解

第14步：从干净会话一键复现并核对逐步结果

这一步解决什么问题

证明逐步路线和一键路线调用同一管道并产生相同输出。

为什么现在运行

Environment 中残留对象可能掩盖依赖错误；最终提交前必须从头运行。

输入对象

当前 RStudio Project 的 `data/bicycle.csv` 和正式代码目录。

可直接运行

```
rm(list = ls())
project_code_root <- "/Users/liumingyuan/Documents/project of stat38888"
source(file.path(
  project_code_root, "R", "handbook_step_by_step.R"
))
result <- run_handbook_steps(
  write_outputs = TRUE,
  workspace_root = getwd()
)
result$model_fit$metrics
```

逐行解释

`rm(list=ls())` 只清空 R 内存对象，不删除磁盘文件。`run\_handbook\_steps()` 按第 1–12 步相同顺序调用正式函数；`workspace\_root=getwd()` 让结果写入当前 `/Users/liumingyuan/sta pj/output`。

运行后应该看到

自动生成审计 CSV、16.800 行面板、预测 CSV、模型指标和 8 张图；指标与第 11 步完全相同。

检查命令

```
stopifnot(nrow(result$panel) == 16800L)
stopifnot(all(c("lon", "lat") %in% names(result$panel)))
stopifnot(nrow(result$model_fit$metrics) == 4L)
list.files(file.path(getwd(), "output", "figures"))
```

常见错误

若清空后失败，说明前面曾依赖未显式生成的对象。不要用旧 Environment 补对象；从错误出现的第一步修正。

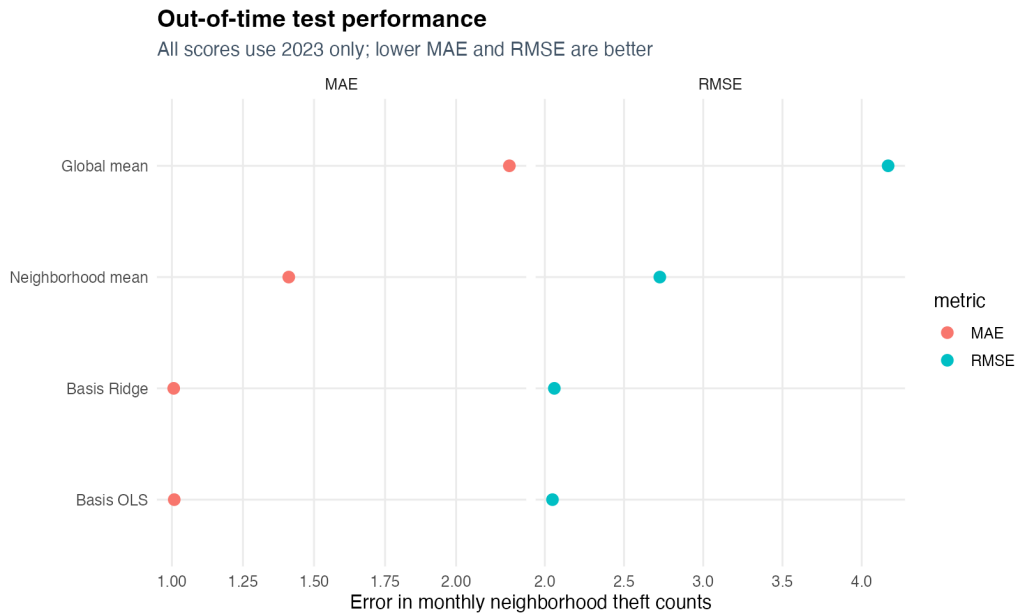
评分连接

逐步与一键结果一致，是本手册修复后的最终可复现性证明。

核心路线

真实模型结果与解释

全局均值	2.19	4.17	-0.007
社区历史均值	1.41	2.73	0.569
基函数OLS	1.01	2.05	0.757
基函数Ridge	1.01	2.06	0.754



2023时间外测试：基函数模型明显优于两个均值基线。

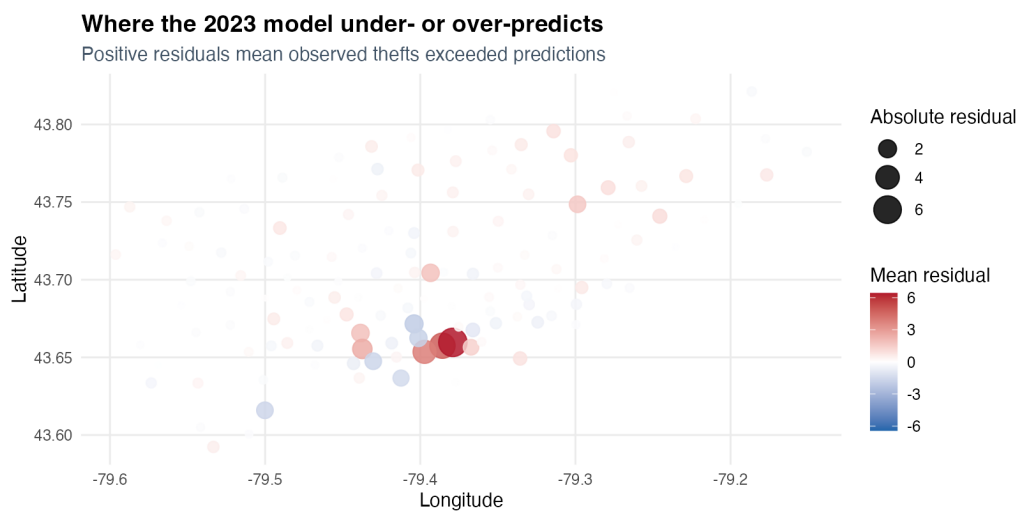
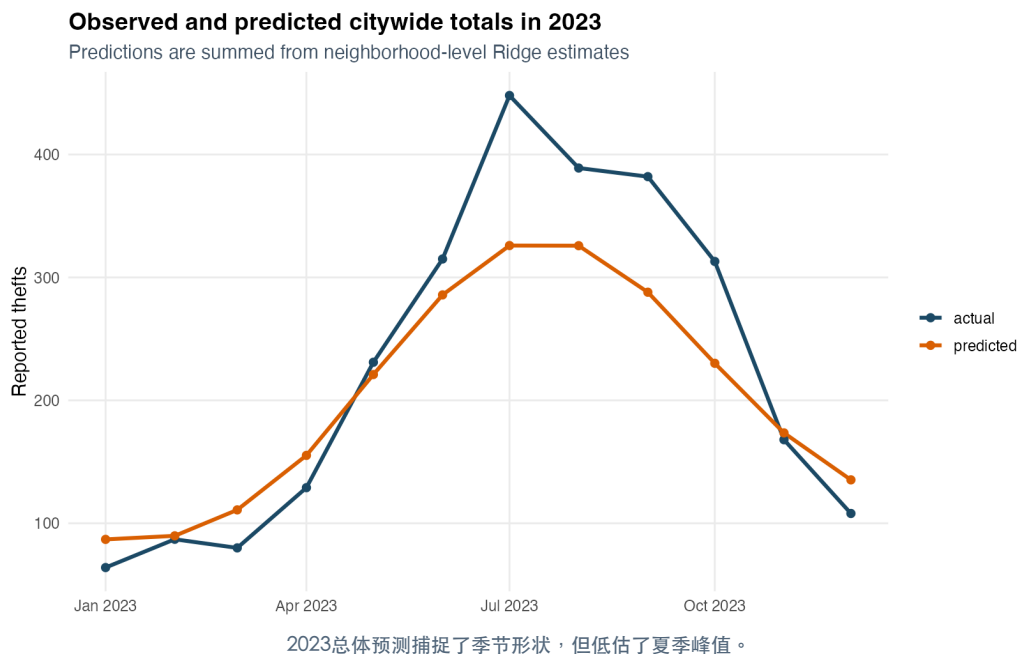
基函数OLS相对社区均值基线把RMSE降低约25%，说明季节、长期趋势和空间/社区结构提供了历史均值之外的信息。Ridge没有进一步提升，说明当前设计矩阵在本任务上已较稳定。

怎样写结论

可以说“模型在2023测试集上具有更低预测误差”，不能说“这些变量导致盗窃”，也不能把测试R²解释为对所有城市和未来年份都同样有效。

进阶加分

预测图、残差图与模型诊断



- 峰值低估是平方误差模型和平滑基函数常见现象。
- 残差仍在中心区域聚集，说明土地用途、交通、人流等遗漏变量可能重要。
- 残差图不只是装饰；它检查模型是否系统性遗漏空间结构。
- 进一步可检查残差随时间、拟合值和社区的分布，但四页PPT只保留最关键证据。

核心路线

四页PPT逐页设计

1	问题与数据	研究问题、31.833事件、140社区、120个月、 workflow
2	Task 1发现	趋势图、热点图、季节性与描述性发现
3	Task 4方法	基函数公式、训练/验证/测试、模型比较
4	预测与结论	预测/残差图、三条结论、局限和下一步

- 每页只回答一个问题，不把所有诊断都塞进展示。
- 字体至少18pt；图标直接写发现，不只写变量名。
- 模型页必须出现简单基线和无泄漏时间切分。
- 结论页必须同时说模型做到了什么、没做到什么。

推荐叙事

看见明显季节与热点 → 用对应基函数编码结构 → 用过去预测2023 → 比基线更准但低估夏季峰值 → 得出有限、可信的结论。

进阶加分

答辩问题与推荐回答

为什么不随机拆分？	会把未来或相邻月份泄漏进训练，误差过于乐观。
为什么 $\log(1+y)$ ？	缓解右偏和极端峰值；预测后还原到计数尺度评价。
为什么既有RBF又有社区指标？	RBF学连续空间，社区指标学稳定但非平滑的地点差异。
Ridge为什么没赢？	验证选择接近零惩罚，说明当前任务无需强收缩；如实报告。
能否预测新社区？	包含社区指标的主模型不能直接外推；需用RBF-only空间模型。
这是因果模型吗？	不是；它描述和预测报告数量，未控制暴露量和混杂因素。

核心路线

局限性、伦理与不可推广范围

- 事件无唯一ID：1,475条完全相同行可能是真实多起事件，也可能含重复记录。
- 坐标是社区中心点：不能解释为案件发生的精确位置。
- 缺少暴露量：高数量可能部分来自人口、自行车拥有量或活动量更高。
- 报案偏差：数据描述被报告事件，不一定覆盖所有真实盗窃。
- 模型只测试2023：不能保证制度、疫情或城市结构变化后的稳定性。
- 热点展示可能造成社区污名化：使用中性语言，不把地点居民描述为原因。

正确措辞

写“报告的自行车盗窃数量在这些社区较高”，不要写“这些社区更危险”或“社区导致盗窃”。

未来连接

从本项目走向未来数据科学

基函数与线性模型	GAM、P-spline、趋势面回归	特征含义与平滑度验证
Ridge与验证	Lasso、Elastic Net、Boosting	超参数只用验证集
社区月份面板	随机森林、XGBoost	时空拆分和泄漏检查
空间RBF	Gaussian Process、Kriging	距离、尺度和不确定性
时间基函数	状态空间、LSTM	过去预测未来
残差地图	空间误差模型、GNN	检查遗漏的结构

真正可迁移的不是某个包名，而是：定义粒度、构造特征、建立基线、隔离测试集、选择指标、诊断残差、限制结论。这些原则在传统统计、机器学习和深度学习中都不会过时。

核心路线

从开始到提交的最终检查清单

- 研究问题能否用一句话说清？
- 一行数据代表什么？时间和空间单位是否明确？
- quarter是否从date重新计算？重复行是否说明而非武断删除？
- 四张Task 1图是否分别回答时间、季节、空间和时空问题？
- 每类基函数是否有明确原因？
- 训练、验证、测试是否严格按时间分开？
- 是否与至少两个简单基线比较？
- MAE、RMSE和 R^2 是否来自2023测试集？
- 是否展示预测和残差，而不只报最好分数？
- 结论是否避免因果化、风险化和过度推广？
- 四页PPT是否每页只有一个中心问题？
- R脚本能否从头运行并重新生成所有图表？

未来连接

资料与继续学习入口

- 原始任务：project.pdf。
- 原始数据：four_dataset/bicycle.csv。
- 本地知识库：矩阵与线性变换、特征值与特征向量、协方差与相关系数、PCA与SVD、R与时空数据入门、ggplot2与空间地图。
- R官方：splines::bs与predict.bs文档。
- R官方：mgcv smooth terms文档。
- tidymodels：sliding_window、sliding_index和sliding_period时间重采样。
- spatialsample：空间分块与leave-location-out验证。

- StatQuest : PCA、回归、正则化和模型评价视频。

学习顺序

先能解释设计矩阵和验证，再深入样条推导；先能复现实验和诊断，再追求更复杂模型。